Deployment and Network Architecture

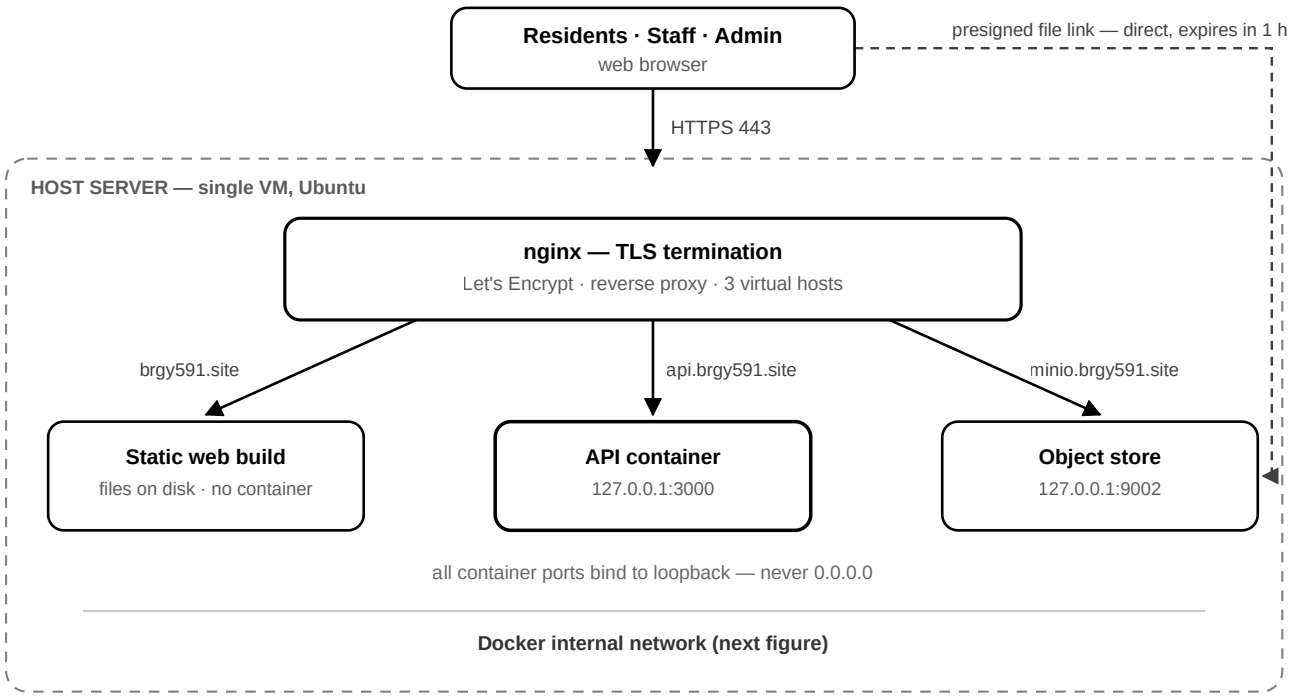
How a resident's browser reaches the system, what sits behind the web server, and where every component runs. One virtual machine, one Docker network, and exactly three addresses exposed to the internet.

Topology single-host, container-per-service **Public surface** 3 vhosts over TLS

AI self-hosted, not a third party

How users reach the system

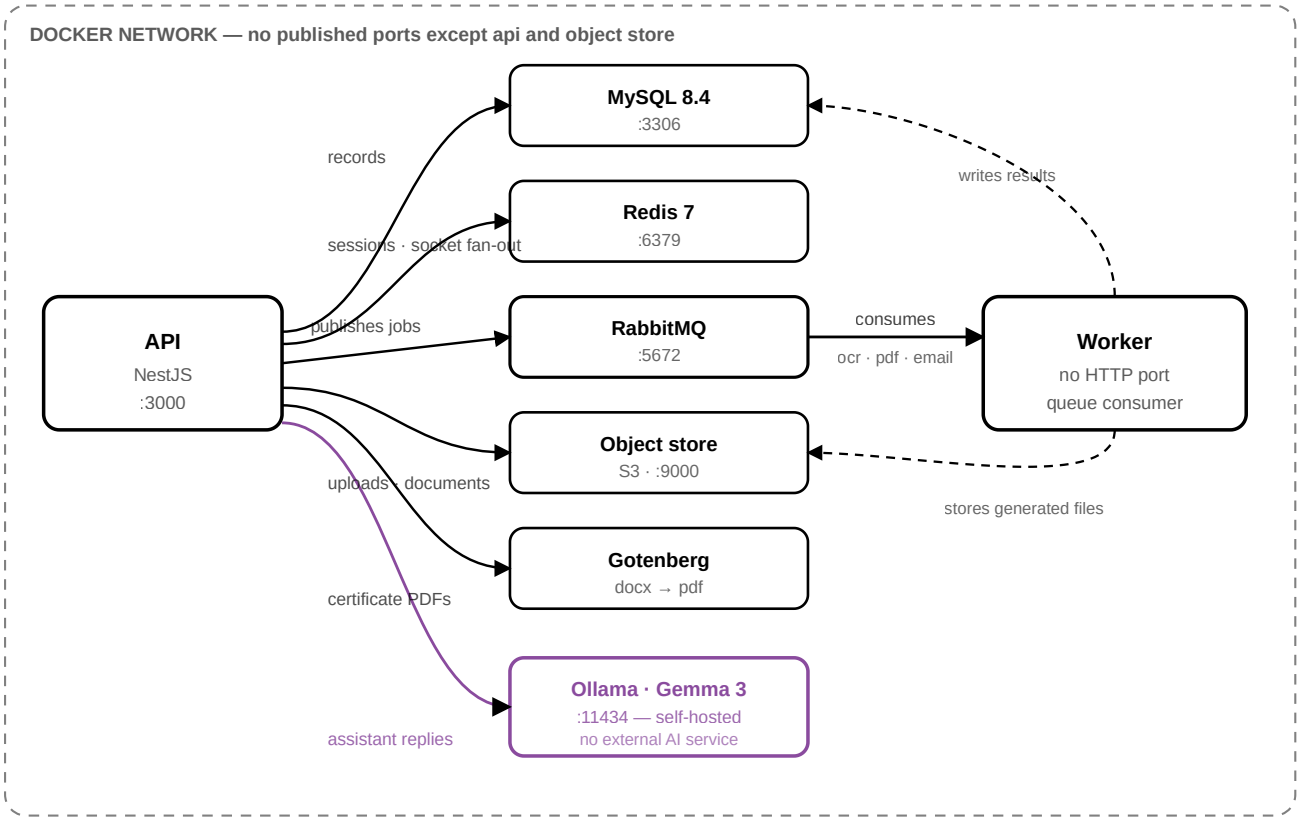
Every request arrives at nginx on the host. Nothing behind it is reachable from the internet — containers publish only to loopback.



Three public addresses, all terminating at nginx. Files are the exception to the API path: the browser fetches them straight from storage using a signed, short-lived URL, so large downloads never pass through the API.

What runs inside

Ten containers on one private Docker network. Only the API and the object store are reachable from nginx; the rest talk to each other by service name.



Solid lines are synchronous calls the API makes while answering a request; dashed lines are the worker writing back after a queued job. The assistant's language model runs in a container on the same machine — no resident data leaves the server.

Exposure summary

What an attacker on the internet can reach, and what they cannot.

COMPONENT	BINDING	REACHABLE FROM
nginx	`0.0.0.0:443`	Internet — the only public listener
API	`127.0.0.1:3000`	nginx only, via `api.brgy591.site`
Object store	`127.0.0.1:9002`	nginx only, via presigned links
Storage admin UI	`127.0.0.1:9003`	SSH tunnel only — never proxied
MySQL · Redis · RabbitMQ	no published port	Docker network only
Gotenberg · Ollama	no published port	Docker network only
Worker	no listener at all	nothing — it only consumes queues

On the AI component

The consultation assumed the assistant was a third-party service. It is not. Ollama runs Gemma 3 in a container on this same machine, with no port published and no outbound call to any AI provider. The practical consequence is that resident questions never leave the server, and the system has no external AI dependency, API key, or per-request cost.

Request walkthrough

A resident requesting a Barangay Clearance, end to end:

- Browser resolves `brgy591.site`, nginx serves the static build.
- The app calls `api.brgy591.site`; nginx forwards to the API on loopback.
- The API authenticates the session, writes the request to MySQL, and publishes a PDF job to RabbitMQ.
- The worker picks the job up, renders the certificate through Gotenberg, and stores the file in the object store.
- The worker queues an email; a second consumer sends it, and a notification goes out over the socket via Redis.
- When the resident downloads, the API returns a presigned link and the browser fetches the file **directly from storage** — the one path that bypasses the API.

Drawn from the production compose file and the nginx site configuration rather than from intent, so the diagram reflects what actually deploys.